

CEUB Music – Streaming de Música

Modelagem Inicial de Persistência em Bancos de Dados NoSQL

GRUPO
Centauros

CAMPUS
Taguatinga

DATA
29/09/2026

DISCIPLINA
NoSQL & Big Data

1. Descrição do Problema

1.1 Contexto da Aplicação

O **CEUB Music** é uma plataforma de streaming musical voltada para a comunidade universitária do CEUB (estudantes, professores e colaboradores). Inspirado em sistemas modernos como Spotify e Apple Music, o sistema suporta quatro operações fundamentais em tempo real:

1. Sessão de Reprodução Ativa

Armazenamento ultraeficiente da faixa atual, posição em segundos e estado do player (play/pause). Dados voláteis que exigem respostas sub-milissegundos.

2. Histórico de Reproduções

Log persistente de todas as execuções (*scrobbles*) por usuário. Base para feeds recentes, relatórios individuais e algoritmos de recomendação.

3. Rankings em Tempo Real

Listagens dinâmicas ("Top Em Alta", "Top Gênero") atualizadas instantaneamente a cada reprodução, alimentando as telas principais da plataforma.

4. Gestão de Playlists

Criação e ordenação de listas personalizadas. Leitura em bloco com baixa latência e mutações leves (adicionar/remover/reordenar músicas).

1.2 Requisitos Principais & Volumetria Estimada

A arquitetura foi dimensionada para a escala operacional do primeiro ano na comunidade acadêmica:

Métrica / Requisito	Estimativa Operacional	Impacto Arquitetural
Base de Usuários Ativos	8.000 usuários cadastrados	Necessidade de isolamento e partição por usuário.
Catálogo de Músicas	500.000 faixas cadastradas	Consultas rápidas e diretas por chave única de faixa.
Reproduções Diárias (Méd / Pico)	40.000 méd. / 80.000 no pico	Picos concentrados nos intervalos de aula e almoço.
Vazão de Escrita (Pico)	~200 escritas simultâneas/s	Inviabiliza locks transacionais em bancos relacionais.
Crescimento do Histórico	~14,6 milhões de logs/ano (~2,9 GB)	Exige banco Wide-Column orientado a append distribuído.

Desafio do Pico de Carga: Mais de 60% do tráfego diário ocorre em janelas de 15 minutos (9h50–10h05 e 12h00–12h30). O sistema precisa suportar explosões de escrita sem degradação do player.

2. Modelagem Inicial dos Dados

Adotou-se uma arquitetura de **Persistência Poliglota**, combinando a família **Chave-Valor (Redis)** para dados voláteis e computação em memória, e a família **Wide-Column (Cassandra)** para o histórico durável de alto volume.

2.1 Dados em Chave-Valor — Redis EM MEMÓRIA

A. Sessão Ativa (Hash + TTL)

Armazena o estado do player. Expira automaticamente em 2h de inatividade (*soft state*).

```
KEY: sessao:usuario:5021
HASH fields:
  musica_atual -> "track:88201"
  posicao_seg   -> 147
  status       -> "playing"
TTL: 7200s
```

B. Fila de Reprodução (List)

Estrutura FIFO para gerenciar as próximas faixas da fila do usuário.

```
KEY: fila:usuario:5021
LIST items:
[0] "track:44102"
[1] "track:77503"
[2] "track:91204"
```

C. Rankings em Tempo Real (Sorted Set)

Score incrementado via **ZINCRBY** a cada play. Busca Top 50 em $O(\log N)$ com **ZREVRANGE**.

```
KEY: ranking:global:diario:2026-09-29
ZSET:
  track:88201 -> score: 1427
  track:44102 -> score: 1183
```

D. Playlists e Contadores (Hash & String)

Metadados da playlist e contadores de plays globais atômicos via **INCR** sem race conditions.

```
KEY: playlist:usuario:5021:rock
HASH: { nome: "Rock", faixas: 25 }
KEY: plays:track:88201 -> 234567
```

2.2 Dados em Wide-Column — Apache Cassandra QUERY-FIRST

A. Histórico de Reproduções por Usuário

Modelado estritamente para atender à consulta: "Listar as últimas *N* músicas ouvidas pelo usuário *X*".

```
CREATE TABLE historico_por_usuario (
  usuario_id      int,
  reproduzido_em  timestamp,
  track_id        text,
  nome_musica     text,
  artista         text,
  duracao_seg     int,
  PRIMARY KEY ((usuario_id), reproduzido_em)
) WITH CLUSTERING ORDER BY (reproduzido_em DESC);
```

Partition Key (`usuario_id`): Isola todo o histórico do mesmo usuário no mesmo nó do cluster.

Clustering Column (`reproduzido_em DESC`): Grava os dados fisicamente no disco já ordenados cronologicamente.

Leitura instantânea dos registros mais recentes sem custos de ordenação.

B. Analytics de Reproduções por Artista (Desnormalização Orientada a Consulta)

```
CREATE TABLE reproducoes_por_artista (
```

2.3 Diagrama do Fluxo de Arquitetura de Dados

Abaixo ilustra-se como uma única ação de reprodução (evento "Play") aciona simultaneamente as duas camadas de persistência sem bloquear a execução do cliente:



3. Justificativa Tecnológica (Checklist do Professor)

3.1 Unidade 1: Fundamentos de Sistemas Distribuídos

Gargalos e Limites do Modelo Relacional

1. **Custo de Joins Sob Carga:** Gerar o "Top Músicas" no SQL exige `JOIN` entre Tabela_Plays, Músicas e Artistas com `GROUP BY` e `COUNT(*)`. Sob 200 escritas/s, essa consulta analítica satura o I/O do servidor.
2. **Contenção de Locks em Contadores:** Inserir `UPDATE faixas SET plays = plays + 1` gera concorrência severa de linhas de tabela.
3. **Escala Vertical Limitada:** O histórico de 14,6M logs/ano exigiria upgrade contínuo de hardware relacional centralizado.

Teorema CAP: Opção AP

O CEUB Music prioriza **Disponibilidade (A)** e **Tolerância a Partição (P)**. O player não pode falhar por indisponibilidade. Se o ranking estiver atrasado em 5 segundos, a experiência do usuário não é impactada.

Modelo BASE vs. ACID

Adoção do modelo **BASE**:

- **Basically Available:** Falha de nó não derruba o player.
- **Soft State:** Sessões expiram sintonizadas via TTL.
- **Eventually Consistent:** Convergência de réplicas em milissegundos.

Estratégia de Sharding e Replicação

Tecnologia	Estratégia de Sharding	Estratégia de Replicação
Cassandra	Hash-based pelo Murmur3 Partitioner usando <code>usuario_id</code> como Partition Key. Distribuição uniforme sem hotspots.	Master-Master (Peer-to-Peer): Fator de Replicação RF=3. Sem ponto único de falha; qualquer nó aceita escritas.
Redis	Consistent Hashing via Redis Cluster dividido em 16.384 hash slots. Minimiza re-hashing ao adicionar nós.	Master-Slave + Redis Sentinel: Leituras escaladas nas réplicas e failover automático do Master em caso de queda.

3.2 Unidade 2: Chave-Valor (Redis) & Dynamo Parallel

A aplicação da família Chave-Valor no CEUB Music reflete exatamente os princípios do **Amazon Dynamo** e do **Redis**:

- **Uso das 5 Estruturas Nativas:** Hashes (sessão e playlists), Lists (fila de reprodução), Sorted Sets (rankings em tempo real em $O(\log N)$), Strings (contadores atômicos com [INCR](#)) e TTL (limpeza automática de soft state).
- **Consistent Hashing:** Mapeamento de chaves no anel hash do Redis Cluster, permitindo escalar a infraestrutura durante horários de pico (intervalos de aula) adicionando nós sem remanejamento massivo de dados.

3.3 Unidade 3: Wide-Column (Cassandra)

Diferente da modelagem relacional orientada a entidades (ER), no Cassandra aplicou-se a filosofia **Query-First**:

- **Column Families Orientadas a Consulta:** Cada tabela CQL responde exclusivamente a um padrão de acesso. Para suportar relatórios por usuário e relatórios por artista, os dados são intencionalmente **desnormalizados** e gravados duplicados nas duas tabelas. No NoSQL, o custo de disco é residual perto do ganho de vazão.
- **Anatomia da Chave Primária:**
 - [Partition Key](#) : Determina o nó físico responsável por guardar a partição (ex.: [usuario_id](#)).
 - [Clustering Column](#) : Define a ordenação física dentro do SSTable no disco (ex.: [reproduzido_em DESC](#)), transformando pesquisas de histórico recente em leituras sequenciais puras.

4. Matriz Resumo da Arquitetura NoSQL

Caso de Uso	Modelo NoSQL	Tecnologia	Mecanismo / Estrutura Chave	Garantia CAP
State do Player	Chave-Valor	Redis	Hash com TTL de 7200 segundos	AP (Soft State)
Fila do Player	Chave-Valor	Redis	List (FIFO com LPUSH / RPOP)	AP (Baixa latência)
Top "Em Alta"	Chave-Valor	Redis	Sorted Set (ZINCRBY / ZREVRANGE)	AP (Consistência Eventual)
Contador Plays	Chave-Valor	Redis	String (Operação atômica INCR)	AP (Sem Locks)
Histórico / Logs	Wide-Column	Cassandra	PK: usuario_id, CK: reproduzido_em DESC	AP (Escrita Peer-to-Peer)
Painel Artista	Wide-Column	Cassandra	PK: artista_id, CK: reproduzido_em DESC	AP (Query-First / Desnormalizado)

5. Referências Bibliográficas

- SADALAGE, P. J.; FOWLER, M. **NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence**. Addison-Wesley, 2013.
- DeCANDIA, G. et al. **Dynamo: Amazon's Highly Available Key-Value Store**. SOSP, 2007.
- CHANG, F. et al. **Bigtable: A Distributed Storage System for Structured Data**. OSDI, 2006.
- LAKSHMAN, A.; MALIK, P. **Cassandra: A Decentralized Structured Storage System**. LADIS, 2010.
- REDIS DOCUMENTATION. **Redis Data Structures & Cluster Architecture**. Disponível em: <<https://redis.io/docs/>>. Acesso em: 2026.